

PowerRAG: Building Smarter and More Accurate RAG Systems with OceanBase's Multi-modal Retrieval

Quanqing Xu
xuquanqing.xqq@oceanbase.com
OceanBase, Ant Group
Hangzhou, China

Chuanhui Yang*

Feizhi Cai
feizhi.cfz@oceanbase.com
OceanBase, Ant Group
Hangzhou, China

Li Feng
yanlou.fl@oceanbase.com
OceanBase, Ant Group
Hangzhou, China

Jiannan Ji
jiannan.jjn@oceanbase.com
OceanBase, Ant Group
Hangzhou, China

Zhifeng Yang
zhuweng.yzf@oceanbase.com
OceanBase, Ant Group
Beijing, China

Guoping Wang
guoping.wgp@oceanbase.com
OceanBase, Ant Group
Hangzhou, China

Yi Zhang
yuanzhi.zy@oceanbase.com
OceanBase, Ant Group
Shenzhen, China

Abstract

To reduce hallucinations and improve LLM response quality, the Retrieval-Augmented Generation (RAG) paradigm has emerged. RAG retrieves relevant content from external knowledge sources conditioned on the user's query, then concatenates the query with the retrieved context as model input (an augmented prompt) to obtain the LLM response. We propose PowerRAG, an RAG-based intelligent application development engine. PowerRAG leverages query rewriting, multi-format document preprocessing, RAPTOR-based document indexing, self-developed caching, and candidate retrieving and reranking powered by a proprietary high-performance database, effectively mitigating hallucinations caused by outdated parametric knowledge and gaps in long-tail knowledge. Using general-purpose intelligent chat assistant as example scenarios, we demonstrate the efficiency and comprehensive features of building AI applications with PowerRAG, as well as its ease and speed during the use of AI applications, contributing our efforts to the further improvement of the Agentic RAG ecosystem.

CCS Concepts

• **Information systems** → **Information retrieval**; *Database management system engines*; *Search engines*.

Keywords

Retrieval-Augmented Generation, Hierarchical Indexing, Hallucination Mitigation, Hybrid Retrieval, Knowledge Graph

ACM Reference Format:

Quanqing Xu, Chuanhui Yang, Feizhi Cai, Li Feng, Jiannan Ji, Zhifeng Yang, Guoping Wang, and Yi Zhang. 2026. PowerRAG: Building Smarter and More Accurate RAG Systems with OceanBase's Multi-modal Retrieval. In *Companion of the International Conference on Management of Data (SIGMOD Companion '26)*, May 31–June 05, 2026, Bengaluru, India. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3788853.3801589>

*Chuanhui Yang is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGMOD Companion '26, Bengaluru, India*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2450-3/2026/05
<https://doi.org/10.1145/3788853.3801589>

1 Introduction

The fundamental capabilities of large language models (LLMs) have realized continuous advancement. Subsequently, issues of LLMs such as hallucinations and outdated knowledge are receiving increasing attention. Retrieval-Augmented Generation (RAG) can effectively mitigate these problems. RAG is a technical framework that combines information retrieval with generative language models. The retriever obtains documents or information snippets relevant to a user's query from external knowledge bases (such as databases, document collections, or vector stores), while the generator uses these retrieved results as context to produce the final answer. RAG can significantly enhance the timeliness, scalability, and reliability of the knowledge generated by LLMs: (1) updates to the knowledge base take effect immediately, (2) the knowledge base is independent of the large model and supports quick additions, deletions, modifications, and queries, (3) data sources can be traced and verified, and (4) there is no need to fine-tune the model, thereby reducing model costs.

HyDE [1] first generated hypothetical answers and then used them for semantic retrieval, further improving retrieval matching. FiD [2] demonstrated that retrieved results could be concatenated and then fed to the decoder to generate the answer. Subsequently, Atlas [3] proposed a pre-trained retrieval-augmented language model that supports a wide range of tasks by learning from few examples in prompts. During retrieval, RAPTOR [4] surfaces information in multiple granularities, from fine-grained chunks to high-level topic overviews, by querying the hierarchical tree-organized index. It selects the most semantically similar nodes across levels as results, thereby improving retrieval matching accuracy. ClueRAG [5] presented a graph-based RAG method that features a multi-partite graph index integrating textual chunks, knowledge units, and semantic entities.

Building on these surprising foundations, we introduce PowerRAG, a solution that tightly integrates RAG, powered by a proprietary high-performance vector retrieval and full-text search engine core, and equipped with agentic orchestration capabilities to efficiently deliver a knowledge-driven intelligent application. Assuming the private/domain knowledge corpus is ready, PowerRAG first processes the data and builds a full-text search index. We then split the documents into chunks (e.g., 1024 tokens/chunk) and use an embedding model to generate dense vectors for each chunk, constructing a hierarchical (tree) index based on RAPTOR along with a vector database. OceanBase [6] serves as the core backend

for vector search and full-text retrieval. For each query, PowerRAG performs query preprocessing and then converts the query into a query embedding. Next, it runs hybrid retrieval – combining full-text search with vector search – followed by re-ranking and summarization (optional) on the retrieved candidates. The final candidates are concatenated into a prompt, and the LLM produces the final response based on this prompt. Given that retrieval may return low-quality or weakly relevant context, PowerRAG supports the ability for self-correction of LLM. We depict above components of PowerRAG in Figure 1. At the same time, we design an adaptive caching mechanism for PowerRAG to store valuable information that may be reused throughout the pipeline, in order to reduce RAG inference latency (details in § 2.3).

2 System Overview

In Figure 1, there are several significant modules of PowerRAG, including but not limited to query preprocessing, multi-format document preprocessing, RAPTOR-based document summarization and indexing, and candidate retrieval and reranking powered by our self-developed high-performance engine. Figure 2(a) demonstrates that PowerRAG maintains the same performance or even surpasses that of RagFlow in comprehensive RAG metrics. Figure 2(b) shows performances comparing PowerRAG with RAG enabled versus disabled across different common datasets.

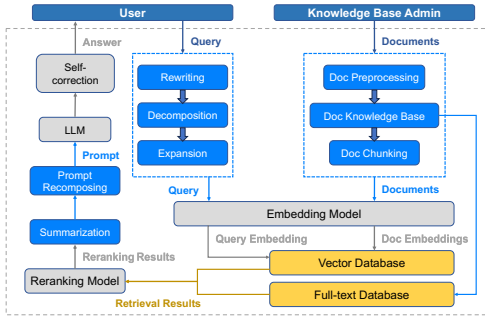
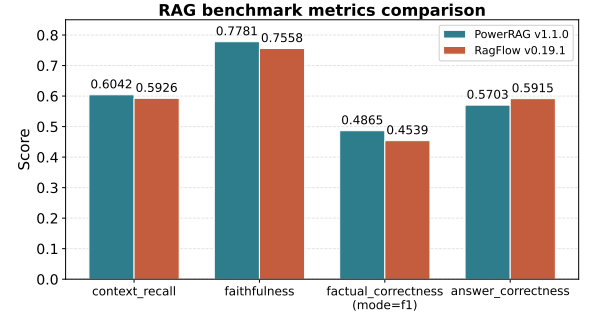


Figure 1: Overview of PowerRAG

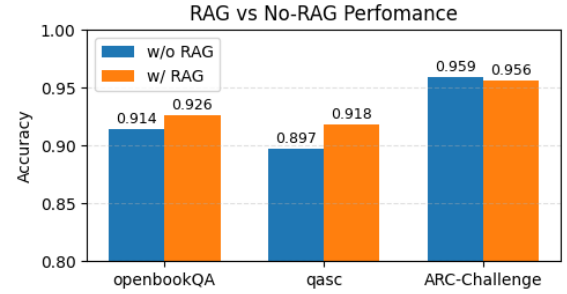
2.1 Document Preprocessing

Multi-format Document Parsing. PowerRAG’s document preprocessing pipeline applies modality-aware parsing strategies tailored to paragraph text, tables and images. In Figure 3, for plain-text paragraphs, if the text requires scanning, it uses OCR to detect and extract the text, otherwise it performs newline (line-break) detection to recover the original line structure. The text is then chunked. For in-document images, we typically perform cropping followed by chart-type classification, route each chart type to specialized processing models, and finally chunk the processed outputs. For tabular data, PowerRAG performs table structure recognition, extracts the cell contents and linearizes them into coherent natural-language statements, and then applies chunking.

RAPTOR-based Retrieval. To balance the gap between fine-grained details and document-level/global context in document chunks, we introduce RAPTOR [4]. RAPTOR uses a tree structure to store both high-level overviews and low-level details about a text. Similarly, we build a bottom-up “abstract tree”. After segmenting



(a) Comparison of different frameworks



(b) Comparison of different datasets

Figure 2: Performance evaluation of PowerRAG.

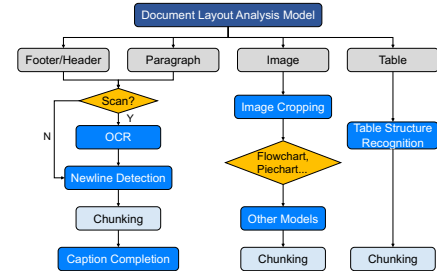


Figure 3: Document parsing of PowerRAG

the texts into chunks, we compute embeddings and perform semantic clustering. We summarize each cluster to produce parent nodes for the underlying chunks. We then embed, cluster, and summarize these same-level summaries again to induce higher-level nodes, recursing until a hierarchical tree is formed. At retrieval time, we adopt *Collapsed Tree* strategy: flatten the hierarchy and rank all nodes across levels by cosine similarity; subject to a maximum prompt budget (e.g., 2,000 tokens), select the most relevant nodes as the retrieved results. By RAPTOR, we preserve fine-grained evidence while achieving clear gains on topic-level, multi-hop queries (e.g., book-level understanding or overview-style QA).

2.2 Query Preprocessing

Query Transformation. After the user submits a question, the query first undergoes query rewriting, query decomposition, and query expansion. Query rewriting performs coreference disambiguation, normalizes abbreviations and aliases, and may extract

salient keywords. Query decomposition breaks a complex question into retrievable sub-questions to enable multi-step retrieval and evidence aggregation, commonly via iterative multi-query retrieval or parallel multi-query retrieval. Query expansion supplements synonyms, variants, and related concepts to narrow the query-document semantic gap.

Hypothetical Document Embeddings (HyDE). PowerRAG leverages HyDE [1] to realize an alternative form of query expansion and reduce the mismatch between queries and answers/evidence.

Inspired by the HyDE methodology, we first generate a hypothetical answer document from the user’s query. This hypothetical document is then embedded into a vector, which is used to retrieve real documents from the corpus that are closest in semantic similarity. Although this hypothetical document may not be factually accurate, it effectively captures the intent behind the query. HyDE’s hypothetical documents tend to resemble actual answers more closely, making it easier to match relevant documents — often by including clearer entities, structure, and context.

2.3 Retrieval and Postprocessing

Self-developed Retrieval Engine. After query transformation, the new generated query is sent to the retriever to obtain relevant knowledge. PowerRAG uses our OceanBase database for the storage and retrieval of both textual data and vector embeddings. In addition, we have integrated the RAPTOR-based tree-structured retrieval paradigm into OceanBase’s vector search. OceanBase is a fully self-developed, high-performance, highly available database system. It supports BM25, dense-vector and sparse-vector retrieval, and provides multiple vector distance metrics including Manhattan (L1), inner product (IP) and cosine similarity.

Postprocessing. After obtaining results from hybrid search, we feed the retrieved candidates into a reranking model for fine-grained ordering. The reranker scores the relevance of the query–candidate pair and promotes highly similar results to the top. Considering that some documents may be long and could degrade response quality and latency, we can summarize these documents/passages first and then concatenate the summaries with the original user query to form a knowledge-augmented prompt for the LLM, which then generates answers according to the prompt.

Caching. To further reduce the first-token latency in PowerRAG applications, we design a caching mechanism in Fig 4. Observing that most queries frequently recall certain specific documents, our RAG pipeline dynamically caches the text segments and associated KV cache data of these hot documents during service, thereby eliminating substantial redundant computation when these documents are recalled again. In addition, we also cache the retrieval plans generated by the LLM prior to retrieval process. For subsequent similar queries, this bypasses the LLM’s plan generation and the following retrieval stage, significantly shortening the overall reasoning chain and noticeably reducing the end-to-end latency of the RAG system.

2.4 Knowledge Graph (KG)

As an optional capability in PowerRAG, Knowledge Graph combined with LLM can automatically parse interpersonal relationships and event timelines from docs, perform entity and relation

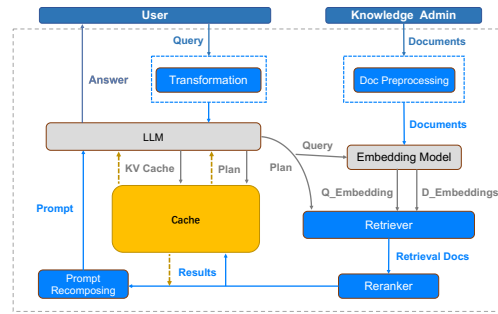


Figure 4: PowerRAG cache mechanism

extraction, and materialize them as a graph. Graph-based Retrieval-Augmented Generation (GraphRAG) represents a significant evolution of traditional RAG: by incorporating a KG to models and traversing complex inter-entity relationships, it substantially improves LLM performance on complex queries, cross-entity relationship, multi-hop reasoning, and global semantic understanding.

3 Demonstration Scenario

Our scenario demo will be presented from two perspectives: the Agent app builder and the assistant’s end user. From the builder’s perspective, we focus on how users can leverage PowerRAG to rapidly and conveniently assemble the desired capabilities using low-code. From the end-user perspective, we focus on the operational flow when using applications of PowerRAG and the high performance user experience. We will demonstrate a general-purpose intelligent chat assistant built on PowerRAG, which is expected to answer common daily questions, such as encyclopedic QA, code implementation, and document understanding. Almost all the capabilities required to build AI apps can be achieved by freely composing PowerRAG’s natively implemented, feature-rich components, highlighting the convenience and efficiency of app construction.

3.1 Demo Architecture

Our general-purpose intelligent assistant is organized into three primary modules. First, the preprocessing module is responsible for acquiring foundational yet critical information – for example, intent classification, current timestamp, and multi-format document parsing – as shown in Figure 5. This stage is composed by orchestrating classification models, conditional routing, and code executor components, with parallel execution across components to minimize end-to-end latency.

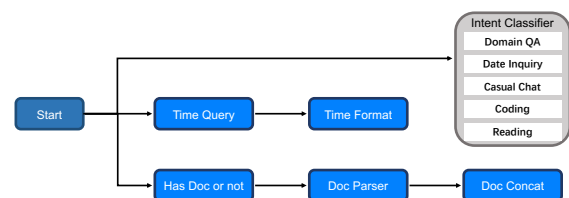


Figure 5: The first module: Preprocessing

Next is the core answering logic module. For different query types, PowerRAG supports rapidly composing flexible processing

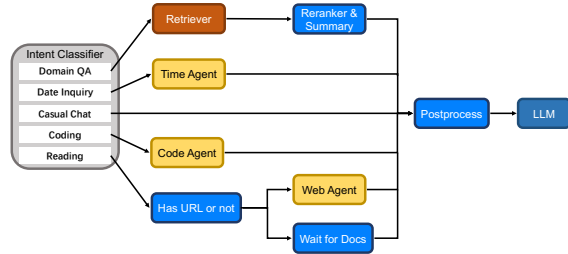


Figure 6: The second and third modules

pipelines. For knowledge-oriented QA, builders can provision an encyclopedic knowledge base that supports both full-text (e.g., BM25) and vector retrieval, with optional automatic knowledge graph construction. After retrieving the most relevant top k documents for the user’s query, a filtering and reranking stage removes low-quality or low-relevance items, then summarizes and orders the remaining context. For document understanding queries, the system first detects whether the user provided a file upload or a URL. For uploaded files, it directly uses the document content extracted by the first module; for URLs, it parses the link, crawls the page, performs content extraction, and summarizes the webpage.

The third module is post-processing and response generation. The output of different pipelines is normalized and aggregated in the context of the LLM. The model then delivers focused, responsible, and evidence-based answers based on these results, reducing hallucinations and improving response quality. Figure 6 illustrates the response logic module and the post-processing & response generation modules.

3.2 Interactive Demonstration

When users expect the intelligent assistant to serve them, just like other common products on the market, they only need to enter their request or question in the dialog box, and the assistant will try to help them. To handle complex relational reasoning or multi-hop questions more effectively, users can choose to enable the knowledge graph feature. Figure 7 shows PowerRAG automatically extracting Greek mythology story provided by the user and constructing a relatively complete knowledge graph (KG).

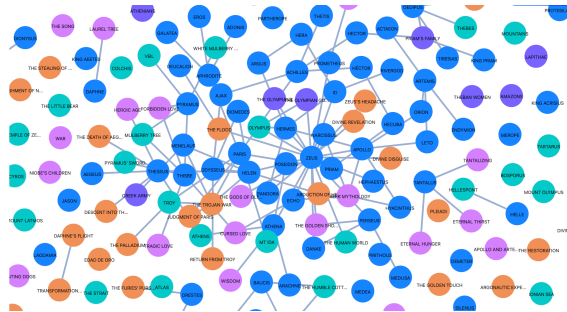


Figure 7: Knowledge graph example

In Figures 8 and 9, we asked what connection there was between Perseus, the slayer of the famous monster Medusa, and Helen, who became a catalyst of the Trojan War. In fact, both are children of Zeus, King of Olympus. Without the KG, the model could not

proactively discover such wide-span relationships and thus failed to give the correct answer. With the graph enabled, the model reasoned based on the graph information and successfully provided the correct answer.

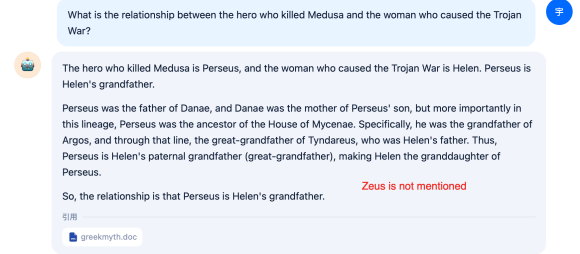


Figure 8: Query without knowledge graph

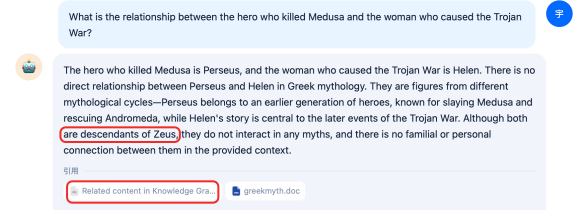


Figure 9: Query with knowledge graph

4 Conclusion

We have developed an intelligent application development engine, PowerRAG, which integrates multiple relevant functional plugins to support one-stop, rapid construction of AI applications. In addition, we designed a comprehensive preprocessing pipeline for both documents and queries to ensure that LLMs can address user needs as accurately as possible. By demonstrating the process of building and using a general-purpose intelligent assistant, we showcased the convenience, efficiency, and comprehensive functionality of PowerRAG. Through the integration of hybrid retrieval, RAPTOR, and knowledge graph technologies, PowerRAG effectively enhances the answer quality of RAG systems, improves timeliness and factual accuracy, and significantly reduces model hallucinations.

References

- [1] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1762–1777.
- [2] Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume*. 874–880.
- [3] Gautier Izacard, Patrick Lewis, Maria Lomeli, et al. 2023. Atlas: few-shot learning with retrieval augmented language models. *J. Mach. Learn. Res.* 24, 1, Article 251 (Jan. 2023), 43 pages.
- [4] Parth Sarthi, Salman Abdullah, Aditi Tuli, et al. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. *arXiv:2401.18059 [cs.CL]* <https://arxiv.org/abs/2401.18059>
- [5] Yaodong Su, Yixiang Fang, Yingli Zhou, et al. 2025. Clue-RAG: Towards Accurate and Cost-Efficient Graph-based RAG via Multi-Partite Graph and Query-Driven Iterative Retrieval. *arXiv preprint arXiv:2507.08445* (2025).
- [6] Zhifeng Yang, Quanqing Xu, Shanyan Gao, et al. 2023. OceanBase Paetica: A Hybrid Shared-Nothing/Shared-Everything Database for Supporting Single Machine and Distributed Cluster. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 3728–3740.